

Revisiting ReDoS: A Rough Idea of Data Exfiltration by ReDoS and Side-channel Techniques (English translation)

Revisiting ReDoS: A Rough Idea of Data Exfiltration by ReDoS and Side-channel Techniques - @speakerdeck, Takashi Yoneuchi, Speaker Deck.

- Published: 2020-02-05
- Original: <https://speakerdeck.com/lmt_swallow/revisiting-redos-a-rough-idea-of-data-exfiltration-by-redos-and-side-channel-techniques>
- Preserved from: https://speakerdeck.com/lmt_swallow/revisiting-redos-a-rough-idea-of-data-exfiltration-by-redos-and-side-channel-techniques (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content (translated into English)

Machine translation of [2020-speaker-deck-revisiting-redos-rough-idea-data-exfiltration-techniques.md](#), which holds the source's own words. Code, payloads, type names, URLs and CVE identifiers were masked before translating and restored after, so they are byte-identical to the original.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Revisiting ReDoS: A Rough Idea of Data Exfiltration by ReDoS and Side-channel Techniques - Speaker Deck

Revisiting ReDoS: A Rough Idea of Data Exfiltration by ReDoS and Side-channel Techniques

Update: you can find a blog article on this presentation at: <https://diary.shift-js.info/blind-regular-expression-injection/>

I presented about ReDoS and blind regular expression injection attack at OWASP Night (Japan) 2020. Please feel free to contact <https://twitter.com/y0n3uchy> if you have questions or find something wrong.

about me: <https://shift-js.info>

[image: Avatar for Takashi Yoneuchi]

Takashi Yoneuchi

February 05, 2020

More Decks by Takashi Yoneuchi

[See All by Takashi Yoneuchi](#)

[
Creating a “Common Language” for Product Security — Using Technical Education and Policy as Code as Examples / "Language" for Product Security

](https://speakerdeck.com/lmt_swallow/language-for-product-security)

[[\[image: Avatar for Takashi Yoneuchi\]](#) lmt_swallow](https://speakerdeck.com/lmt_swallow)

0

960

[
Mastering Security Engineering with SRE / SRE, Security Engineering, and You

](https://speakerdeck.com/lmt_swallow/sre-security-engineering-and-you)

[[\[image: Avatar for Takashi Yoneuchi\]](#) lmt_swallow](https://speakerdeck.com/lmt_swallow)

5

3.6k

[
Eliminating ReDoS with Ruby 3.2

](https://speakerdeck.com/lmt_swallow/eliminating-redos-with-ruby-3-dot-2)

[[\[image: Avatar for Takashi Yoneuchi\]](#) lmt_swallow](https://speakerdeck.com/lmt_swallow)

0

390

[
The Future of the Software Supply Chain / Securing Software Supply Chain

](https://speakerdeck.com/lmt_swallow/securing-software-supply-chain)

[[\[image: Avatar for Takashi Yoneuchi\]](#) lmt_swallow](https://speakerdeck.com/lmt_swallow)

1

360

[
Accelerating AWS Security Management with Policy as Code — Toward the Best CSPM Experience / Unleashing Policy as Code on AWS CSPM

](https://speakerdeck.com/lmt_swallow/unleashing-policy-as-code-on-aws-cspm)

[[\[image: Avatar for Takashi Yoneuchi\]](#) lmt_swallow](https://speakerdeck.com/lmt_swallow)

4

2.6k

[

Practical SpiceDB — Toward Implementing Permission Management That Can Survive the Cloud-Native Era / Practical SpiceDB

](https://speakerdeck.com/lmt_swallow/practical-spicedb)

[[\[image: Avatar for Takashi Yoneuchi\]](#) lmt_swallow](https://speakerdeck.com/lmt_swallow)

0

2.2k

[

The Concept of Developer-First Security / Introduction to Developer-First Security

](https://speakerdeck.com/lmt_swallow/introduction-to-developer-first-security)

[[\[image: Avatar for Takashi Yoneuchi\]](#) lmt_swallow](https://speakerdeck.com/lmt_swallow)

6

10k

[

Let's Build a Small Web Browser (Online Lecture Version) / Build Your Own Web Browser

](https://speakerdeck.com/lmt_swallow/build-your-own-web-browser)

[[\[image: Avatar for Takashi Yoneuchi\]](#) lmt_swallow](https://speakerdeck.com/lmt_swallow)

18

12k

[

Let's Build "Guardrails" for Writing Secure Go / Let's Build Security Guardrails For Your Go Programs!

](https://speakerdeck.com/lmt_swallow/lets-build-security-guardrails-for-your-go-programs)

[[\[image: Avatar for Takashi Yoneuchi\]](#) lmt_swallow](https://speakerdeck.com/lmt_swallow)

4

6.1k

Other Decks in Research

[See All in Research](#)

[

Computational Informatics Laboratory (Mathematical Informatics Laboratory No. 7) 2026

](https://speakerdeck.com/tomohirokoana/ji-suan-qing-bao-xue-yan-jiu-shi-shu-li-qing-bao-xue-di-7yan-jiu-shi-2026)

[[\[image: Avatar for Tomohiro\]](#) tomohirokoana](https://speakerdeck.com/tomohirokoana)

0

720

[
Model Discovery and Graph Simulation: A Lightweight Gateway to Chaos Engineering
(https://speakerdeck.com/anatolykr/model-discovery-and-graph-simulation-a-lightweight-gateway-to-chaos-engineering)

[[\[image: Avatar for Anatoly A. Krasnovsky\]](#) anatolykr](https://speakerdeck.com/anatolykr)

0

250

[
PHTalks Bengaluru - SSRF When All Else Fails

(https://speakerdeck.com/dk999/phtalks-bengaluru-ssrf-when-all-else-fails)

[[\[image: Avatar for Debangshu Kundu\]](#) dk999](https://speakerdeck.com/dk999)

0

940

[
The Work of a Research Engineer / Research Engineering: Bridging Research and Business
(https://speakerdeck.com/chck/research-engineering-bridging-research-and-business)

[[\[image: Avatar for chck\]](#) chck](https://speakerdeck.com/chck)

1

250

[
GLIM and MegaParticles: The Limits of Gaussian Approximation and Advances in Tight Coupling & Particle Filters / GLIM and MegaParticles : Progress of the distribution representation in SLAM
(https://speakerdeck.com/koide3/glim-and-megaparticles-progress-of-the-distribution-representation-in-slam)

[[\[image: Avatar for koide3\]](#) koide3](https://speakerdeck.com/koide3)

0

680

[

Proposal and Demonstration of a Joint AS Operation Model Toward Establishing Autonomous IPv6 Connectivity at the Laboratory Level

](https://speakerdeck.com/reokashiwa/as_dojo_ipv6)

[ reokashiwa](https://speakerdeck.com/reokashiwa)
PRO

0

190

[
SAKURAONE:An Open Ethernet-based AI HPC System And Its Observed Workload Dynamicsin a Single-Tenant LLM Development Environment

](https://speakerdeck.com/yuukit/sakuraone-an-open-ethernet-based-ai-hpc-system-and-its-observed-workload-dynamics-in-a-single-tenant-llm-development-environment)

[ yuukit](https://speakerdeck.com/yuukit)

1

500

[
Nationwide Town/Aza-Level Vacant House Rate Estimation Data ver1.0 Specifications

](https://speakerdeck.com/microbaseinc/quan-guo-ting-zi-dan-wei-kong-kijia-lu-tui-ding-detaver1-dot-0detashi-yang)

[ microbaseinc](https://speakerdeck.com/microbaseinc)

0

170

[
[CV Study Group @ Kanto CVPR2026] PSDesigner: Automated Graphic Design with a Human-Like Creative Workflow / kantocv 67th CVPR 2026

](https://speakerdeck.com/shunk031/kantocv-67th-cvpr-2026)

[ shunk031](https://speakerdeck.com/shunk031)

0

220

[
Discussion: What Does It Take to Make an Academic Movement Successful?

](https://speakerdeck.com/rmaruy/yi-lun-xue-shu-mubumentowocheng-gong-saserutamenihe-gabi-yao-nanodarouka)

[ rmaruy](https://speakerdeck.com/rmaruy)

0

110

[
How Much of SLAM Has Been Solved?
(https://speakerdeck.com/tomonom/robomech26-workshop-print)
[[image: Avatar for tomonom](#)] tomonom (https://speakerdeck.com/tomonom)
0
1k
[
Scalable dynamic origin-destination demand estimation enhanced by high-resolution satellite imagery data
(https://speakerdeck.com/satai/scalable-dynamic-origin-destination-demand-estimation-enhanced-by-high-resolution-satellite-imagery-data)
[[image: Avatar for SatAI.challenge](#)].png) satai (https://speakerdeck.com/satai)

3

380

Featured

[See All Featured](#)

[
The AI Search Optimization Roadmap by Aleyda Solis
(https://speakerdeck.com/aleyda/the-ai-search-optimization-roadmap-by-aleyda-solis)
[[image: Avatar for Aleyda Solis](#)] aleyda (https://speakerdeck.com/aleyda)
1
6k
[
Digital Projects Gone Horribly Wrong (And the UX Pros Who Still Save the Day) - Dean Schuster
(https://speakerdeck.com/uxyall/digital-projects-gone-horribly-wrong-and-the-ux-pros-who-still-save-the-day-dean-schuster)
[[image: Avatar for UX Y'all](#)] uxyall (https://speakerdeck.com/uxyall)
1
2.2k
[
Put a Button on it: Removing Barriers to Going Fast.
(https://speakerdeck.com/kastner/put-a-button-on-it-removing-barriers-to-going-fast)

[[\[image: Avatar for kastner\]](#) kastner](<https://speakerdeck.com/kastner>)

60

4.5k

[

Optimising Largest Contentful Paint

](<https://speakerdeck.com/csswizardry/optimising-largest-contentful-paint>)

[[\[image: Avatar for Harry Roberts\]](#) csswizardry](<https://speakerdeck.com/csswizardry>)

37

3.9k

[

Dealing with People You Can't Stand - Big Design 2015

](<https://speakerdeck.com/cassininazir/dealing-with-people-you-cant-stand-big-design-2015>)

[[\[image: Avatar for Cassini Nazir\]](#) cassininazir](<https://speakerdeck.com/cassininazir>)

367

27k

[

What Awaits Us in a World Where AI Writes 90% of the Code? / What awaits us in a world where 90% of the code is written by AI

](<https://speakerdeck.com/rkaga/what-awaits-us-in-a-world-where-90-percent-of-the-code-is-written-by-ai>)

[[\[image: Avatar for r-kagaya\]](#) rkaga](<https://speakerdeck.com/rkaga>)

63

45k

[

The MySQL Ecosystem @ GitHub 2015

](<https://speakerdeck.com/samlambert/the-mysql-ecosystem-at-github-2015>)

[[\[image: Avatar for Sam Lambert\]](#) samlambert](<https://speakerdeck.com/samlambert>)

251

13k

[

From π to Pie charts

](<https://speakerdeck.com/rasagy/from-p-to-pie-charts>)

[[\[image: Avatar for Rasagy Sharma\]](#) rasagy](<https://speakerdeck.com/rasagy>)

0

250

[

How to Align SEO within the Product Triangle To Get Buy-In & Support - #RIMC

](https://speakerdeck.com/aleyda/how-to-align-seo-within-the-product-triangle-to-get-buy-in-and-support-number-rimc)

[[\[image: Avatar for Aleyda Solis\]](#) aleyda](https://speakerdeck.com/aleyda)

2

1.8k

[

Producing Creativity

](https://speakerdeck.com/orderedlist/producing-creativity)

[[\[image: Avatar for Steve Smith\]](#) orderedlist](https://speakerdeck.com/orderedlist)

PRO

348

40k

[

Bioeconomy Workshop: Dr. Julius Ecuru, Opportunities for a Bioeconomy in West Africa

](https://speakerdeck.com/akademiya2063/bioeconomy-workshop-dr-julius-ecuru-opportunities-for-a-bioeconomy-in-west-africa)

[[\[image: Avatar for AKADEMIYA2063\]](#) akademiya2063](https://speakerdeck.com/akademiya2063)

PRO

1

190

[

Impact Scores and Hybrid Strategies: The future of link building

](https://speakerdeck.com/tamaranovitovic/impact-scores-and-hybrid-strategies-the-future-of-link-building)

[[\[image: Avatar for Tamara Novitovic\]](#) tamaranovitovic]

(https://speakerdeck.com/tamaranovitovic)

0

370

Transcript

-

Revisiting ReDoS Takashi Yoneuchi (@y0n3uchy) Flatt Security, Inc. Department of

Information Science, Faculty of Science, The University of Tokyo A Rough Idea of Data Exfiltration by ReDoS and Side-channel Techniques

-

Takashi Yoneuchi **Twitter** **ja:@lmt_swallow** **en:@y0n3uchy** **Affiliation:**

Flatt Security, Inc. Department of Information Science, Faculty of Science, the University of Tokyo I <3 Web Leader of @ctf4b Staff of @security_camp, #websecjp Member of TSG / dodododo (CTF team) See: <https://shift-js.info>

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Outline Introduction to Algorithmic Complexity Attack Definition and examples Regular Expression Denial of Service (ReDoS) 101 Rough sketch of implementations of regexp engines Definition and examples of ReDoS Mitigation of ReDoS (Maybe) New Idea: Blind Regular Expression Injection Attack Explanation of Blind Regular Expression Injection Attack 3

-

Introduction to Algorithmic Complexity Attack

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Computational Complexity Preliminaries for the introduction to AC Attack Computational complexity for an algorithm is the amount of resources required for running it. Time complexity: the amount of the required time Space complexity: the amount of the size of the memory There are two kinds of computational complexity. Average-case complexity Worst-case complexity Examples: searching in a binary search tree (BST) The average-case time complexity: $O(\log n)$. The worst-case time complexity: $O(n)$. 5

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Algorithmic Complexity Attack A low-bandwidth DoS attack A security aspect of algorithms: worst-case complexity. To prevent algorithmic complexity attacks, we have to care about worst-case complexity of an algorithm as well as average-case one. Algorithmic Complexity Attack: DoS by worst-case inputs In 2003, Crosby and Wallach proposed a class of DoS attacks by giving a crafted worst-case input for applications. S. A. Crosby and D. S. Wallach, "Denial of Service via Algorithmic Complexity

Attacks," in Proceedings of the 12th Conference on USENIX Security Symposium - Volume 12, 2003, p. 3. This class of attacks may cause a DoS with a small input; in other words, this is a low-bandwidth DoS attack. The class of low-bandwidth DoS attacks is often called asymmetric DoS (ADOS). 6

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Real-world Examples Algorithmic Complexity Attack There are a lot of real-world examples. The followings are a part of them: "Hash-flooding DoS reloaded: attacks and defenses" at 29C3

https://131002.net/siphash/siphashdos_appsec12_slides.pdf "I Came to Drop Bombs: Auditing the Compression Algorithm Weapon Cache" at BlackHat USA 2016

<https://www.blackhat.com/docs/us-16/materials/us-16-Marie-I-Came-to-Drop-Bombs-Auditing-The-Compression-Algorithm-Weapons-Cache.pdf> "Denial of Service with a Fistful of Packets: Exploiting Algorithmic Complexity Vulnerabilities" at BlackHat USA 2019

<https://www.blackhat.com/us-19/briefings/schedule/#denial-of-service-with-a-fistful-of-packets-exploiting-algorithmic-complexity-vulnerabilities-16445> 7

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Academical Efforts For Algorithmic Complexity Attack By-hand Exploration: Crosby et al., 2003 (mentioned before), Cai et al., 2009, Sun et al., 2011, ... X. Cai, Y. Gui, and R. Johnson, "Exploiting Unix File-System Races via Algorithmic Complexity Attacks," in 2009 30th IEEE Symposium on Security and Privacy, 2009, pp. 27–41. X. Sun, L. Cheng, and Y. Zhang, "A Covert Timing Channel via Algorithmic Complexity Attacks: Design and Analysis," in 2011 IEEE International Conference on Communications (ICC), 2011, pp. 1–5. (Semi-) Automated Detection: Tools by Holland et al., 2016, SlowFuzz (Petsios et al., 2017), Badger (Noller and Kersten, 2018), ... T. Petsios, J. Zhao, A. D. Keromytis, and S. Jana, "SlowFuzz: Automated Domain-Independent Detection of Algorithmic Complexity Vulnerabilities," in Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2155–2168. Y. Noller, R. Kersten, and C. S. Păsăreanu, "Badger: Complexity Analysis with Fuzzing and Symbolic Execution," in Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2018, pp. 322–332. B. Holland, G. R. Santhanam, P. Awadhutkar, and S. Kothari, "Statically-Informed Dynamic Analysis Tools to Detect Algorithmic Complexity Vulnerabilities," in 2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation (SCAM), 2016, pp. 79–84. 8

ReDoS 101

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Regular Expressions a.k.a. regex, regexp
Regular expressions (a.k.a. regex, regexp) are powerful and useful pattern matching language for strings. Two major security aspects of regexp: correctness and performance. Weak validation by incomplete (or incorrect) regular expressions. Example: `preg_replace("/on/", "", $input)` for detecting event handlers A lot of possible bypasses: `oNload`, `Onload`. ref. "Regex Security Cheatsheet" <https://github.com/attackercan/regexp-security-cheatsheet> Too heavy computations. This causes Regular Expression Denial of Service; ReDoS. 10

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Implementation of RE Engines Convert regexp into NFA and simulate it Fact: Every regular expression has an equivalent Non-deterministic Finite Automaton (NFA) and vice versa. Thompson's Algorithm: regexp → NFA. Kleene's Algorithm: NFA → regexp. Implementation: After converting regexp into NFA (or DFA), ... 1. Choose one among possible next states and backtrack when it failed to match. 2. Choose all of them and continue the simulation simultaneously. K. Thompson, "Regular expression search algorithm," Comm. ACM, vol. 11, no. 6, pp. 419–422, 1968. <https://www.fing.edu.uy/inco/cursos/intropln/material/p419-thompson.pdf> R. Cox, "Regular Expression Matching Can Be Simple And Fast (but is slow in Java, Perl, PHP, Python, Ruby, ...) " <https://swtch.com/~rsc/regexp/regexp1.html> 11

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Backtracking (1) In the case of NFA-based engines Let an input for the regexp `^a+a+$` (the concatenation of two `a+`) be `"aaaaa!"`. First `a+` can match `"a"`, `"aa"`, ..., and `"aaaaa"`. 12 Input: `a a a a a !`

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Backtracking (2) In the case of NFA-based engines Backtracking-based engines chooses one of the candidates (e.g. `"aaaaa"`) and continues to match. When `"aaaaa"` was chosen, the second `a+` cannot match. 13 Input: `a a a a a !`: string matched for the first `a+`) (

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Backtracking (3) In the case of NFA-based engines Then the engines retries with another candidate and continues to match. This behavior is called backtracking. When `"aaaa"` was chosen, the second `a+` can match `"a"`. 14 Input: `a a a a a !`: string matched for the first `a+`) (

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Catastrophic Backtracking What super-linear (SL) regex cause Problem: There are regular expressions that require a lot of backtracking (catastrophic backtracking). $\wedge a+a\$ \dots O(n^2)$ for aaaaa....aaaaa! $\wedge (.+)+a\$ \dots O(2^n)$ for aaaaa ... aaaaa! They require non-linear time in length of an input for evaluation! Impact: a lot of RE engines adopt backtracking-based approach! Python, Node.js, Ruby, etc. 15 import timeit for i in range(0, 30): code = "import re; re.match(r'\w+(.+)a\$', '{}!')".format('a' * i) print(i, timeit.timeit(code, number=1)) Example of catastrophic backtracking

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques ReDoS Regular Expression Denial of Service Attackers can use a lot of computational resources of a server when ... An (Web) application use those backtracking-based RE engines. Attackers can control inputs for a vulnerable regular expression (= a super- linear regular expression) that is pre-defined or crafted by RE injection. This issue is called Regular Expression Denial of Service (ReDoS). Especially, ReDoS has a big impact on Node.js-based applications. Due to its single-threaded nature and backtracking-based RE engine! A great deal of ReDoS vulnerabilities npm modules are reported in 2019. ref. "ReDoS vulnerabilities in npm spikes by 143% and XSS continues to grow" by snyk <https://snyk.io/blog/redos-vulnerabilities-in-npm-spikes-by-143-and-xss-continues-to-grow/> 16

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Prevalence Is ReDoS a popular issue? ReDoS vulnerabilities are in the news! Academical Survey: a large-scale analysis on ReDoS vulnerabilities and reported a lot of possible ReDoS vulnerabilities. C.-A. Staicu and M. Pradel, "Freezing the Web: A Study of ReDoS Vulnerabilities in JavaScript-based Web Servers," in 27th USENIX Security Symposium (USENIX Security 18), 2018, pp. 361–376. J. C. Davis, C. A. Coghlan, F. Servant, and D. Lee, "The Impact of Regular Expression Denial of Service (ReDoS) in Practice: An Empirical Study at the Ecosystem Scale," in Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2018, pp. 246–256. Real-world Examples: a lot of CVEs! Google "ReDoS CVE". 17

-

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Mitigation How can we mitigate ReDoS? There are three major approaches to mitigate ReDoS vulnerabilities. Abort the evaluation of heavy regular expressions. .NET provides optional regex timeouts. <https://docs.microsoft.com/en-us/dotnet/api/system.text.regularexpressions.regex.matchtimeout?view=netframework-4.8> Python's "regex" module (not a built-in one) provides the timeout too. <https://pypi.org/project/regex/> Use non-backtracking engines. For instance, RE2 guarantees

linear-time performance. "RE2 was designed and implemented with an explicit goal of being able to handle regular expressions from untrusted users without risk."

<https://github.com/google/re2/wiki/WhyRE2> Avoid anti-patterns. Do NOT nest quantifiers, Do NOT repeat same patterns with quantifiers, ... 18

Blind Regular Expression Injection Attacks

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Assumption For Blind Regular Expression Injection We assume the following conditions: A victim application evaluates a regexp with a secret. Attackers can control the regexp with using ... the valid feature of applications (e.g. string matching in searching feature). unsafe construction of a regexp (i.e. regular expression injection). Attackers can know or guess the set of characters that might be included in the secret.

Example: An application takes a regexp to search a records in secrets. After evaluating regexp for all possible records, it checks whether an user has the permission to see the search results. 20

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Test case An example of a vulnerable application 21 This (pseudo-) Python script takes a regexp for `secret` as an input from a user, returning no information after the search. It is clear that this application has a ReDoS vulnerability due to `re` module, a backtracking-based regexp engine.

```
import re
import sys
secret = "this_is_secret_value"
r = input("Give a regexp to search: ")
_ = timeout(re.match(r, secret), 5)
print("Done. I won't give you search results :P")
```

 Example: a vulnerable application

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Research Question What does the prevention by resource limiting cause? Research Question: If developers handle a regexp under the strict timeouts for prevention, can we utilize this for malicious use? Idea for RQ: ReDoS + side-channel techniques Resource limitation prevents ReDoS vulnerabilities; we can use as many resources (e.g. time, memory, ...) as the limitation multiple times. i.e. Evaluation of RE may cause a change of resource usage without DoS. Resource usage might be observed by side-channel techniques. e.g. the time to evaluate a regexp on the server side can be approximated by round-trip time. If we can construct a regexp whose resource usage changes according to the text to be searched, we can build an oracle for secret records! 22

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Blind RE Injection Attacks A new class of regular expression injection attacks? Under the threat model, attackers can reveal the secret by ... 1. Constructing the following oracle with regexp injection vulnerabilities or its valid features. The oracle receives a proposition on the secret as a regexp. The oracle returns 1 if the proposition holds, otherwise 0. 2. Querying `length_is(n)` to the oracle again and again to get `len(the secret)`. `length_is(n)` ... whether `len(the secret)` is `n` or not. 3. Querying `starts_with(s)` to the oracle repeatedly to get the whole of the secret. `starts_with(s)` ... whether the secret starts with the string `s` or not. I'd like to call this kind of attacks blind regular expression injection attacks. 23

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques 1. Construct a Oracle Predicates on the secret 24 # `length_is(n)` `.{n}$` # `starts_with(s)` `s.*` # `ends_with(s)` `.s$` # `nth_char_is(n, c)` `.{n-1}c`. The following predicates on the secret can be described as regexps. `length_is(n)`: the length of the secret is `n`. `starts_with(s)`: the secret starts with the string `s`. `ends_with(s)`: the secret ends with the string `s`. `nth_char_is(n, c)`: the `n`'th character of the secret is the char `c`. Predicates on the secret

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques 1. Construct a Oracle `redos_if` function 25 # here we assume the secret does not ends with the string "hoge". `def redos_if(prop): return "^(?=){}((.))hoge$".format(prop)` Let `prop` be a proposition on the secret written as a regexp. e.g. `length_is(3)` The evaluation of `^(?=prop)((.))*hoge$` takes .. a lot of time if `prop` holds. little time if `prop` does not hold. Therefore, we can encode the truth value of `prop` into the time needed for the evaluation of `redos_if(prop)`!

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques 1. Construct a Oracle Combining `redos_if` and timing measurement(s) This snippet do the followings: give a regexp `redos_if(prop)` to a victim application. measure how much time it takes for the application to return the response. returns whether the measured time exceeds the threshold or not. We achieved the construction of an oracle that returns whether a proposition (`prop`) on `secret` holds or not! 26 `import time` `def oracle(prop):` `threshold = 1` `prev = time.process_time()` # (request w/ `redos_if(prop)` and wait the response) `return time.process_time() - prev > threshold` Construction of an oracle with `prop`.

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques 2. Leak len(the secret) Blind Regular Expression Injection Let `ub_of_len` be an upper bound of the length of the secret. It can be guessed :P For each `i` in a closed range `[1, ub_of_len]`, we can check whether `length_is(i)` holds or not by the oracle. Querying `length_is(i)` for all `i` in the range reveals the length of the secret. 27 `ub_of_len = 100` `length_is = lambda n: ".{" + str(n) + "}"` for `i` in `range(1, ub_of_len+1)`: if `oracle(length_is(i))` break # `len(the secret) == i` Naive algorithm to leak `len(secret)`

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques 3. Leak the secret Blind Regular Expression Injection Let `S` be the set of possible characters in the secret. For each position of the secret (`= i`) and for each possible character (`= c`, i.e. the element of `S`), we can check whether `nth_char_is(i, c)` holds or not. In this naive way, we can leak the secret by $O(n |S|)$, where `n` is the length of the secret (`length_of_secret`). 28 `secret = ""` `nth_char_is = lambda n, c: ".{" + str(n) + "-1}" + c` for `i` in `range(0, length_of_secret)`: for `c` in `S`: if `oracle(nth_char_is(i, c))`: `secret += c` Naive algorithm to leak the secret

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Optimization by binary search Blind Regular Expression Injection A optimized algorithm with binary search finishes in $O(n \log |S|)$. We can determine the `len(the secret)` by binary-searching among `[1, (upper bound of the length)]`. Similarly, the secret can be leaked by binary-searching among `S` with `nth_char_in`. 29 # `length_in(n, m)` `.{n-m}$` # `nth_char_in(n, S)` # where `s = ".join(S).{n-1}[s].*` (e.g. `.[abc]$` for `S = {a, b, c}`) Predicates on the secret

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Implications What blind regular expression injection attack implies Blind regular expression injection attack requires ... Evaluation of arbitrary regexps in backtracking-based regexp engines. If regexps used in the applications are constant or safely constructed from user- controllable values. Enough number of evaluation. If a malicious regexp may cause DoS, attackers can't fetch enough information to leak the secret. Here is the important thing: Abortion of the evaluation of use-controllable regexps might induce the exploitability by blind regular expression injection attacks, even though the abortion is for ReDoS prevention! 30

© 2020 shift-js.info Revisiting ReDoS: A Rough Idea of Data

Exfiltration by ReDoS and Side-channel Techniques Takeaways Revisiting ReDoS: A Rough Idea of Data Exfiltration by ReDoS and Side-channel Techniques I presented new class of attacks: Blind

Regular Expression Injection Attacks. To the best of my knowledge, this is the first report of this kind of attacks, although it seems to be a CTF-like technique :P I believe that there are some real-world examples. To avoid security issues related to regexp, you should ... Construct your regexp safely. Do NOT use user-controllable regexp with backtracking-based engines! Use non-backtracking engines (e.g. RE2). Resource limitation including timeouts on backtracking-based engines might induce the issue like my report :O Avoid anti-patterns of ReDoS. 31

-

Thank you for listening. Feel free to contact me: @y0n3uchy

(@lmt_swallow) <https://shift-js.info>

Archived from https://speakerdeck.com/lmt_swallow/revisiting-redos-a-rough-idea-of-data-exfiltration-by-redos-and-side-channel-techniques. Rendered offline from the local Markdown copy.